

Travaux Diriger Programmation II Langage C



SMI S4

Pr AIT DAOUD

Pr OUNACER

EXERCICE 1

En utilisant l'allocation dynamique, écrivez un programme qui permet de calculer la somme de tous les nombres impairs de 1 à n.

Solution

- 1ère méthode

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int n,i,*somme;// déclaration de variables n le nombre, i l'indice courant et *somme c'est la somme qu'on veut calculer
    printf("veuillez saisir le nombre n");
    scanf("%d",&n);

    somme= (int*)malloc(sizeof(int)); //Allocation dynamique de la mémoire

    *somme=0; //initialiser somme a zéro

    for(i=1;i<=n;i+=2)
        *somme+=i;
    printf("la somme des nombres impairs de 1 a %d est:%d",n,*somme);
    free(somme); // Libérer la mémoire
}
```

- 2 -ème méthode

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
int main()
{
    int n,i,*somme;

    printf("veuillez saisir le nombre n");
    scanf("%d",&n);

    somme= (int*)malloc(sizeof(int));

    *somme=0;

    for(i=1;i<=n;i++)
        if((i%2)!=0)
            *somme+=i;
    printf("la somme des nombres impairs de 1 a %d est:%d",n,*somme);
    free(somme);
}
```

EXERCICE 2

Réaliser un programme en C qui permet de trouver l'élément le plus grand d'un tableau à l'aide de l'allocation dynamique. Une gestion des erreurs permettra de renvoyer le pointeur NULL en cas d'erreur d'allocation mémoire.

Solution

```
#include<stdio.h>
#include<stdlib.h>
main()
{
    int i,n,max; // declaration de variables (i indice courant, n taille du tableau, max pour le maximum)
    printf("veuillez saisir la taille du tableau:"); //dimension du tableau
    scanf("%d",&n);
    int *t=malloc(sizeof(int));
    // Gestion des erreurs
    if(t==NULL) // Tester si l'adresse est NULL, dans ce cas-là échec de réservation
    {printf("la mémoire n'est pas allouée");
    exit(0);
    }
    else{ // l'adresse de t est non NULL, alors c'est la réussite de la réservation
    //Remplissage du tableau t
    for(i=0;i<n;i++){
        printf("entrer l'élément t[%d]=",i);
        scanf("%d",&t[i]);
    }
    max=*t; //Affecter au max le premier élément du tableau
    //Comparer les autres éléments avec le max. si l'un des éléments est supérieure au max cet élément devient le maximum
    for(i=0;i<n;i++){
        if(max < *(t+i)){
            max=*(t+i);
        }
    }
    printf("le max des éléments du tableau est:%d",max); //Affichage du maximum
    free(t); //Libérer l'espace mémoire alloué
    }
}
```

EXERCICE 3

Ecrire un programme qui demande un tableau de N valeurs entières (N au maximum égal à 30), puis :

Travail à faire

1. Affiche tous les nombres qui se terminent par 5 ainsi que leur nombre.
2. Affiche la fréquence d'un nombre Nbrech dans le tableau (Nbrech à saisir).

Solution

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n=0, i; //déclaration des variable (n pour la taille du tableau et i pour l'incrément des boucles)
    do // boucle conditionner afin d'obliger l'utilisateur à respecter la taille du tableau demandé
    {
        printf("\n taille (<=30):");
        scanf("%d", &n);
    } while (n <= 0 || n > 30); //condition sur la taille du tableau qui doit pas être un nombre négatif et aussi ne pas dépasser 30
    int *tab; // déclaration du tableau
    tab=(int*)malloc(n*sizeof(int)) ; // allocation mémoire du tableau tab ( n*sizeof(int) je multiplie le nombre d'éléments fois
    // l'espace mémoire qui nécessite chaque entier)
    // remplir le tableau avec les valeurs saisie par l'utilisateur
    for (i = 0; i < n; i++)
    {
        printf("\n Tab[%d]:", i + 1);
        scanf("%d", &tab[i]);
    }
    int count = 0; // déclaration et initialisation de la variable count

    for (i = 0; i < n; i++)
    {
        if (*(tab+i) % 10 == 5) //condition sur les valeurs qui se terminent par 5
        {
            if (count == 0)
                printf("\n terminent par 5 : "); // afficher le texte des valeur qui se terminent par 5
            printf("%d\t", *(tab+i));
            count++; // calculer le nombre des valeurs terminent par 5
        }
    }
    if (count > 0)
        printf("\n leur nombre : %d", count); // afficher le nombre des valeurs terminent par 5
    else
        printf("il n'y a pas de nombre qui termine par 5 dans le tableau");
    int nbrech; // déclaration de la variable de fréquence Nbrech
    printf("\n saisir le nombre Nbrech:");
    scanf("%d", &nbrech);
    count = 0; //initialiser la variable count
    for (i = 0; i < n; i++) //boucle pour parcourir le tableau
        if (*(tab+i) == nbrech) //condition sur l'existence du nombre nbrech
            count++; //incrémenter le nombre de fréquence
    if (count > 0)
        printf("\n la fréquence de Nbrech : %d", count);
    else
        printf("Nbrech n'existe pas dans le tableau");
    free(tab); //libération de la mémoire allouée
}
```

EXERCICE 4

Ecrire un programme, qui saisit deux tableaux d'entiers positifs A et B et qui copie alternativement leurs éléments dans un troisième tableau Tab.

Travail à faire

Tableau A :

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Tableau B :

10	20	30
----	----	----

Tableau Tab :

1	10	2	20	3	30	4	5	6	7
---	----	---	----	---	----	---	---	---	---

Solution :

```
#include <stdio.h>
#include <stdlib.h>
main()
{ //déclaration des variable (n et m est respectivement la taille du tableau A et B & i, j, k pour l'incrémentation des boucles)
int i,j=0,k=0, n, m;
int *A,*B,*TAB; //déclaration des tableaux
do{
printf("donner la dimension du premier tableau A");
scanf("%d",&n);
}while(n<=0); //condition sur la taille du tableau qui doit pas être un nombre négatif
A=(int*)malloc(n*sizeof(int)); // allocation dynamique du tableau A
// Boucle pour le remplissage du tableau A
for(i=0;i<n;i++)
{
printf("A[%d]=",i+1);
scanf("%d", A+i);
}

do{
printf("donner la dimension du deuxième tableau B");
scanf("%d",&m);
}while(m<=0); //condition sur la taille du tableau qui doit pas être un nombre négatif
B=(int*)malloc(m*sizeof(int)); // allocation dynamique du tableau B
// Boucle pour le remplissage du tableau B
for(i=0;i<m;i++)
{
printf("B[%d]=",i+1);
scanf("%d", B+i);
}

TAB=(int*)malloc((n+m)*sizeof(int)); // allocation dynamique du tableau TAB
for(i=0;i<m+n;i+=2) // Boucle pour parcourir le tableau TAB
{
if(j<n && k<m) // si l'indice j du tableau A n'a pas dépassé la taille n et l'indice k du tableau B n'a pas dépassé la taille m
{
*(TAB+i)=*(A+j); //tab[i] reçoit A[j]
*(TAB+i+1)=*(B+k); //tab[i+1] reçoit B[k]
j++; //la prochaine itération on doit incrémenter j et k
k++;
}
else { //j atteint la taille du tableau A et l'indice k n'a pas dépassé la taille m du tableau B
```

```
if(k<m){ //compléter le tableau TAB par les valeurs du tableau B
    *(TAB+i)=*(B+k); //tab[i] reçoit B[k]
    *(TAB+i+1)=*(B+k+1); //tab[i+1] reçoit B[k+1]
    k+=2; //incrémenter avec deux pas
}
else { //k atteint la taille du tableau B et l'indice j n'a pas dépassé la taille m du tableau B alors on complète par les valeurs de A
    *(TAB+i)=*(A+j); //tab[i] reçoit A[j]
    *(TAB+i+1)=*(A+j+1); //tab[i+1] reçoit A[j+1]
    j+=2; //incrémenter avec deux pas
}
}
}

//Affichage du tableau A
printf("A= ");
for(i=0;i<n;i++)
{
    printf("%d\t", *(A+i));
}
printf("\n\n");

//Affichage du tableau B
printf("B= ");
for(i=0;i<m;i++)
{
    printf("%d\t", *(B+i));
}
printf("\n\n");

//Affichage du tableau TAB
printf("TAB= ");
for(i=0;i<n+m;i++)
{
    printf("%d\t", *(TAB+i));
}

//Libération de la mémoire pour les trois tableaux
free(A);
free(B);
free(TAB);
}
```

EXERCICE 5

Ecrire un programme qui demande un tableau de N valeurs entières (N au maximum égal à 50), puis :

Travail à faire

1. Remplir dynamiquement un tableau de n éléments.
2. Calculer et afficher la moyenne de ses éléments.
3. Demander à l'utilisateur de rajouter d'autres éléments ou bien de quitter le programme.
4. En cas de rajout de nouveaux éléments, afficher tous les éléments du tableau et recalculer la nouvelle moyenne.

Solution

```
#include <stdio.h>
#include <stdlib.h>
main()
{ //déclaration de variables (i l'indice courant, n la taille, a le nombre des nouveaux éléments à rajouter, reponse qui sera O
dans le cas de rajout et N dans le cas échant, m la moyenne, s la somme et t le tableau des éléments)
int i,n,a;
char reponse;
float m,s=0,*t;
// boucle conditionner afin d'obliger l'utilisateur à respecter la taille du tableau demandé
do{
printf("donner la taille du tableau"); //Saisir la dimension du tableau
scanf("%d",&n);
}while(n<=0 || n>= 50); //condition sur la taille du tableau qui doit pas être un nombre négatif et aussi ne pas dépasser 50
t=(float*)malloc(n*sizeof(float)); //Allocation dynamique du tableau t
//Remplissage du tableau
printf("saisir les elements du tableau \n");
for(i=0; i<n; i++)
{
printf("t[%d]=",i+1);
scanf("%f",&t[i]);
s+=*(t+i); // calculer la somme des éléments saisies par l'utilisateur
}
m=s/n; // calcule de la moyenne
printf("la moyene des elements est:%.2f\n",m); // Affichage de la moyenne
printf("voulez-vous ajouter d'autres elements (O/N)?"); // Demander à l'utilisateur s'il veut rajouter de nouveaux éléments
scanf(" %c",&reponse);

while(reponse=='O'){ // Tant que la réponse est OUI, entrer le nombre à rajouter
printf("Entrer le nombres d'éléments a ajouter");
scanf("%d",&a);
n=n+a; // changement de la taille du tableau
t= (float*)realloc(t,n*sizeof(float)); // modification de la taille du tableau t qui est déjà allouée avec malloc d'où l'utilisation
de realloc
for(i=0; i<n-a; i++)

printf("élément %d:%.2f\n",i+1,*(t+i)); // Affichage des éléments précédents
printf("saisir les nouveaux éléments:\n");
// Saisie des nouveaux éléments
for(i=n-a; i<n; i++){
printf("element %d :",i+1);
scanf("%f",&t[i]);
s=s+*(t+i); // calculer la somme de tous les éléments
}
m=s/n; // recalculer la nouvelle moyenne
```

```
printf("la nouvelle moyenne est : %.2f\n",m); // Affichage de la nouvelle moyenne
printf("voulez-vous ajouter d'autres elements (O/N)?"); // Demander à l'utilisateur s'il veut rajouter de nouveaux éléments
scanf(" %c",&reponse);
}

free(t); // libérer la mémoire du tableau t
}
```


EXERCICE 6

Ecrire un programme qui lit les points de N élèves d'une classe dans un devoir et les mémorise dans un tableau POINTS de dimension N.

Travail à faire

1. Rechercher et afficher :
 - la note maximale,
 - la note minimale,
 - la moyenne des notes.
2. A partir des POINTS des élèves, établir un tableau NOTES de dimension 5 qui est composé de la façon suivante :
 - NOTES[4] contient le nombre de notes supérieur ou égal à 16
 - NOTES[3] contient le nombre de notes supérieur ou égal à 14 et inférieur strictement à 16
 - NOTES[2] contient le nombre de notes supérieur ou égal à 12 et inférieur strictement à 14
 - NOTES[1] contient le nombre de notes supérieur ou égal à 10 et inférieur strictement à 12
 - NOTES[0] contient le nombre de notes inférieur strictement à 10

Solution

```
#include<stdio.h>
#include<stdlib.h>
main()
{//déclaration des variables
  float *points;
  int *notes;
  float som,max,min,moy;
  int i,n;
  do{
    printf("entrer le nombres des élèves (max 50)");
    scanf("%d",&n);
  }while(n<=0 || n>50);//condition pour ne pas saisir un nombre négatif et pour ne pas dépasser la taille maximale
  points=(float*)malloc(n*sizeof(float));//allocation dynamique de la mémoire pour le tableau points
  printf("Saisir les points des élèves:\n");

  for(i=0;i<n;i++)//remplissage du tableau points
  {
    printf("élève n: %d",i+1);
    scanf("%f",points+i);
  }//initialisation des variables MAX et MIN avec la première valeur du tableau
  max=*points;
  min=*points;
  som=0;
  for(i=0;i<n;i++)//boucle afin de parcourir le tableau
  {
    if(*(points+i)>max){ max=*(points+i);}
    if(*(points+i)<min){ min=*(points+i);}
    som+=*(points+i);
  }
  moy=som/n;
  printf("la note maximale est:%.2f\n",max);
  printf("la note minimale est:%.2f\n",min);
  printf("la moyenne des notes est:%.2f\n",moy);
  int nbr16,nbr14,nbr12,nbr10,nbr9;//déclaration et initialisation de variables pour le tableau notes
  nbr16=nbr14=nbr12=nbr10=nbr9=0;
  notes=(int*)malloc(5*sizeof(int));//allocation dynamique de la mémoire pour le tableau points
  for(i=0;i<n;i++)//boucle pour parcourir le tableau notes
```

```
    //statistiques des notes
    if(*(points+i)>=16)
    {
        nbr16++;
    }else if(*(points+i)>=14){ nbr14++;}
    else if (*(points+i)>=12){ nbr12++;}
    else if (*(points+i)>=10){ nbr10++;}
    else {nbr9++;}
    }
*(notes+4)=nbr16;
*(notes+3)=nbr14;
*(notes+2)=nbr12;
*(notes+1)=nbr10;
*(notes+0)=nbr9;
if(*(notes+4)>0) printf("\nle nombre des notes superieur ou egale a 16 est:%d",*(notes+4));
if(*(notes+3)>0) printf("\nle nombre des notes superieur ou egale a 14 et inferieur strictement a 16 est:%d",*(notes+3));
if(*(notes+2)>0) printf("\nle nombre des notes superieur ou egale a 12 et inferieur strictement a 14 est:%d",*(notes+2));
if(*(notes+1)>0) printf("\nle nombre des notes superieur ou egale a 10 et inferieur strictement a 12 est:%d",*(notes+1));
if(*(notes+0)>0) printf("\nle nombre des notes inferieur strictement a 10 est:%d",*(notes+0));

free(points);//libérer la mémoire du tableau points
free(notes);//libérer la mémoire du tableau notes

}
```